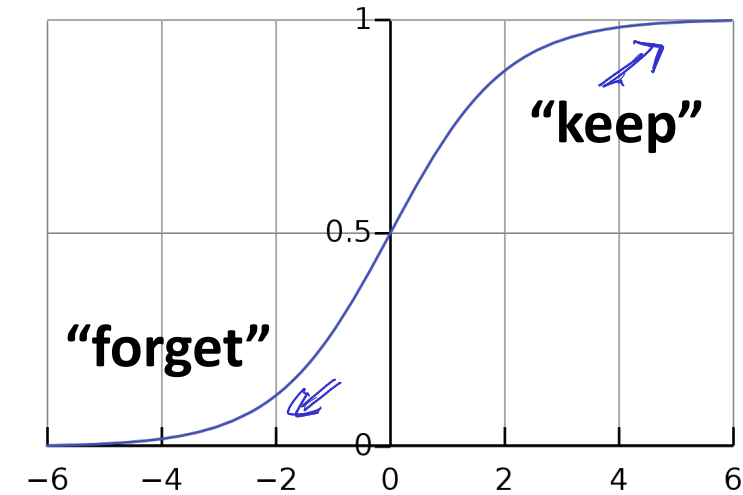
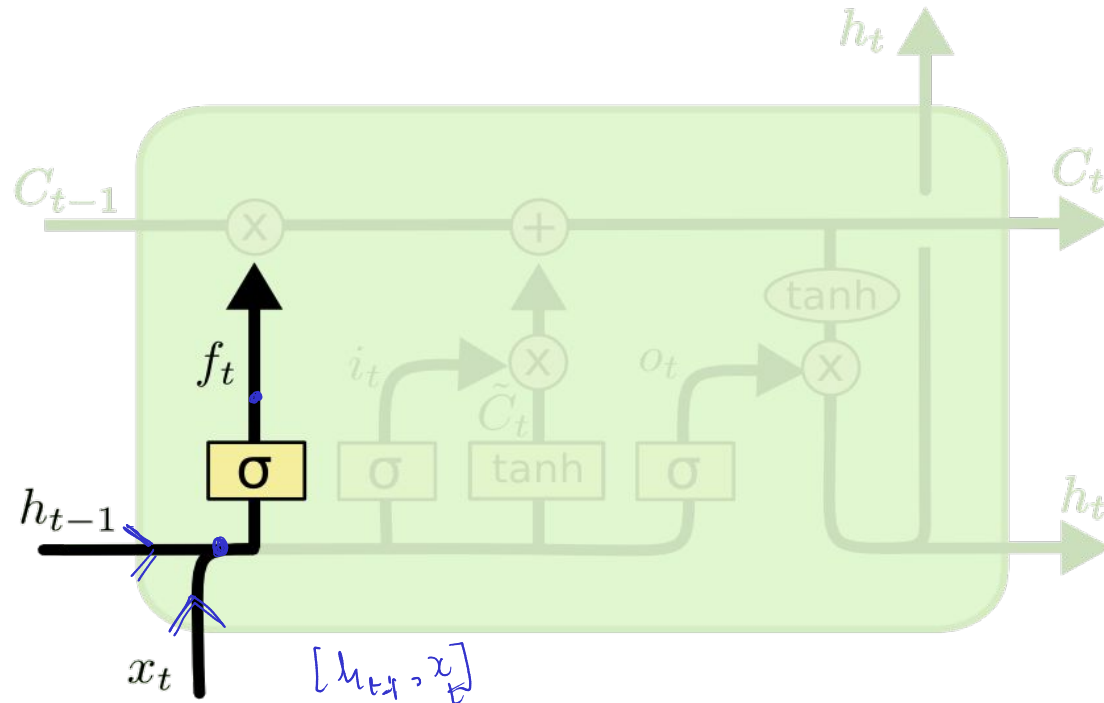


Long-short Term Memory Network

→ cell-state
→ gates =

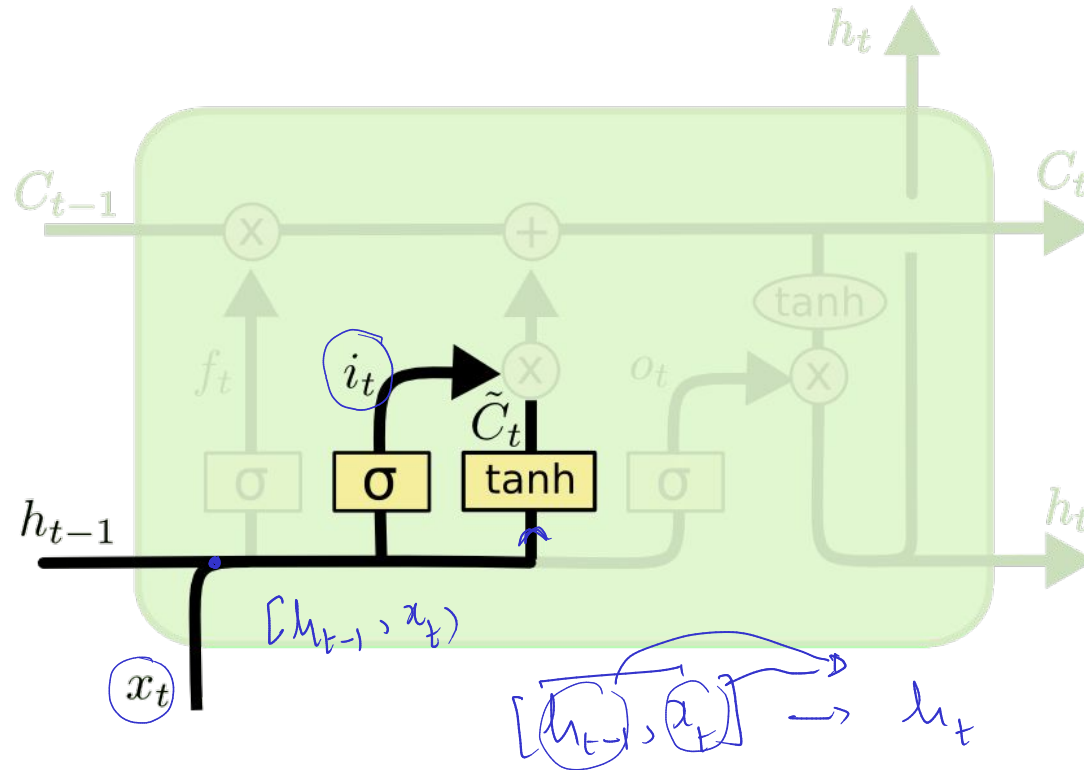
"switch" → learn (remember)
→ forget



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Forget Gate: sigmoid layer decides which information (from previous cell) to remember or discard

Long-short Term Memory Network



switch $\equiv$ selector
 “select which values to update”

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

“create candidate vectors for cell-state”

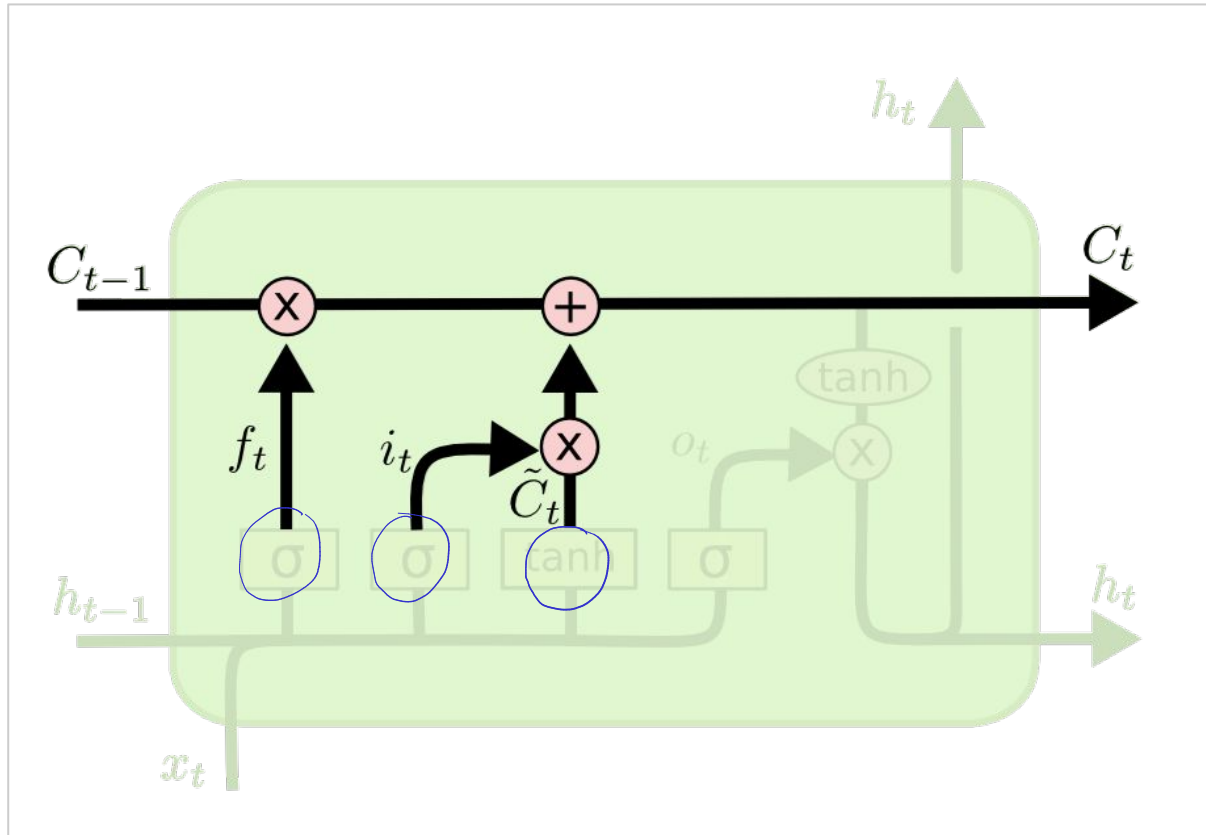
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

↓
“smoother”

Input Gate: selectively updates the cell state based on the new inputs, protect the memory contents stored in hidden state from perturbation by irrelevant inputs

Long-short Term Memory Network

$$RNN \Rightarrow W_{nn}^n +$$



“discard values that are to be forgotten, add the candidate vector, update cell state”

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Forget gate parameters control the gradients such that it does not vanish, and additive property of cell-state gradients enforce that the total gradient is balanced.

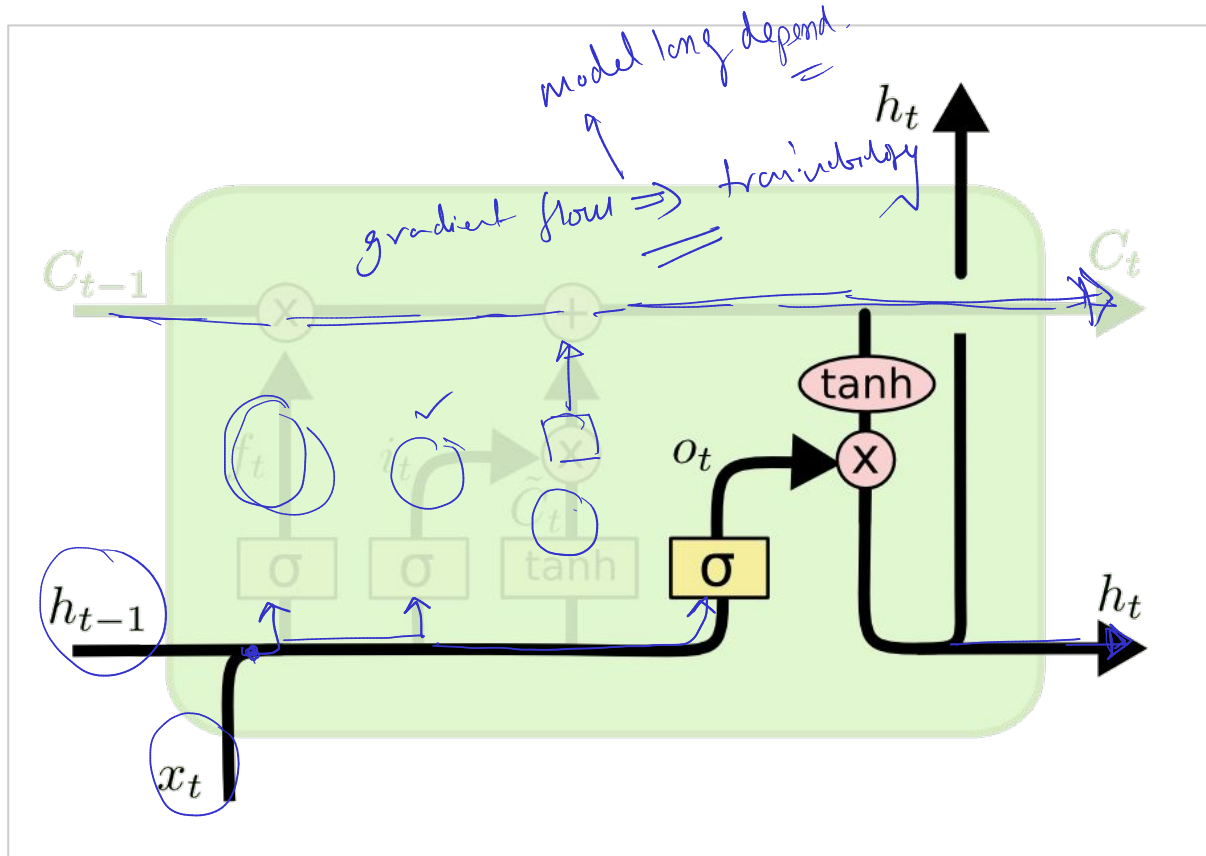
$$\nabla_c = \nabla_f + \nabla_i + \nabla_w$$

$$\nabla \rightarrow 1$$

forget

Cell update: update the cell state based on computations of forget gate and input gate ✓

Long-short Term Memory Network



“output is the filtered version of the cell state”

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Output Gate: multiplicative gate protects output from irrelevant memory contents, hidden state is updated.